SentinelFlow

Behavioral Anomaly Detection for Cybersecurity — Technical Report

Honeywell Hackathon — Problem Statement 4 | 26 July 2026

Bhagya Paras Morabia

Sentinel Flow is a User and Entity Behaviour Analytics (UEBA) system built for Problem Statement 4 of the Honeywell Hackathon: AI-powered behavioural anomaly detection for cybersecurity. The system models what normal access behavior looks like for 600 entities — users, service accounts, and edge devices — drawn from a synthetic access-log dataset, and flags deviations from that baseline in near real time.

Detection runs through three structurally different detectors working in parallel — a deterministic rule engine, a tabular anomaly ensemble, and a sequence model — whose outputs feed a single XGBoost classifier that produces both a 0–100 risk score and a specific attack-type label. Every alert carries a SHAP-based explanation and, where applicable, a MITRE ATT&CK reference. The system is served through a FastAPI backend with a lightweight web frontend for the analyst-facing dashboard.

This report covers the assumptions behind the system's design, the results of evaluating it on a held-out test set, and the limitations we're aware of.

1. Assumptions

1.1 Behavioral Baseline Modeling

The synthetic data generator does not assign random values to each log line. Before generating any events, it builds a stable behavioral profile for every entity — a home geolocation, a typical working-hours pattern, a personal set of resources it normally touches, and a typical session-length distribution. Events are then sampled around that profile.

Working hours are drawn from a von Mises distribution — a circular version of the normal distribution — rather than a plain Gaussian on hour-of-day. A Gaussian centered near midnight would treat 11 PM and 1 AM as far apart, when they're one hour apart on a 24-hour clock. Von Mises handles that wraparound correctly.

Resource access follows a Zipf-weighted catalog: a small number of resources are touched by nearly everyone (shared file servers, common applications), and the rest are personal or team-specific, accessed by only a handful of entities. This mirrors how access patterns look in a real enterprise network, rather than assuming every entity has an equal chance of touching every resource.

Session durations follow a log-normal distribution — most sessions are short, a few run long.

We also deliberately injected noise into the normal data so attacks wouldn't be trivially separable: about 2% of normal sessions include a failed login, so a failed login by itself isn't a reliable signal. Some legitimate entities also occasionally log in from a secondary location — travel, conferences, VPN exit nodes — so unusual geography by itself isn't automatically an attack either.

1.2 Attack Taxonomy

Seven attack patterns are injected into the dataset at controlled rates. Six map to a specific MITRE ATT&CK technique; the seventh, insider_drift, is treated as an edge case rather than a discrete attack (see below).

Vector	Detection Logic	MITRE
brute_force	8 or more failed authentications against the same account from the same source IP within a 10-minute window.	T1110
credential_stuffing	15 or more distinct accounts targeted from a single source IP, with at least 70% of those attempts failing.	T1110.004

Vector	Detection Logic	MITRE
impossible_travel	Two authentications for the same entity, in locations far enough apart that the implied travel speed exceeds 900 km/h.	T1078
lateral_movement	A single session where an entity accesses an unusually high number of resources it has never touched before, right after authenticating.	T1021
device_spoofing	A successful authentication from a known device_id, but with a fingerprint (OS, browser, MAC) that doesn't match that device's history.	T1036
low_and_slow	Small outbound data transfers, sustained over many hours or days, kept deliberately below any single-event volume threshold.	T1048
insider_drift	Gradual change in a trusted account's working hours or resource footprint over a 14–30 day period. Used to tune false-positive tolerance, not scored as a hard target.	—

1.3 Detection Architecture

No single detector catches every attack family well, so instead of one large model, three structurally different detectors run in parallel:

Detector	Purpose	Mechanism
Rule engine	Catch mathematically provable violations instantly, at full precision.	Fixed threshold checks — the same four rules above for brute_force, credential_stuffing, impossible_travel, and device_spoofing — evaluated on every incoming event.
Tabular anomaly ensemble	Catch statistical outliers that don't trip a hard rule.	Isolation Forest and ECOD (both from PyOD), each trained only on normal events, contamination = 0.02. An event's anomaly score is the average of the two models' decision functions. Most useful for device_spoofing and low_and_slow, where the anomaly is a matter of degree.
Sequence model (GRU)	Catch anomalies in the order of actions, not just their volume.	A GRU next-action predictor, DeepLog-style (embedding size 32, hidden size 64, 10-action window), trained to predict an entity's next action from its last 10. If the actual next action isn't in the model's top 5 predictions, the sequence is flagged. Main signal for lateral_movement.

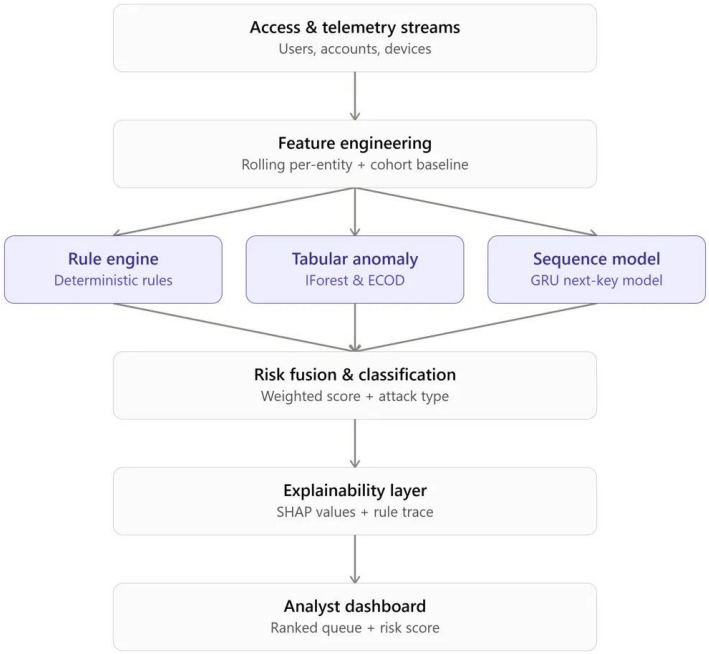

Figure 1 — Detection pipeline: telemetry into feature engineering, three parallel detectors, fusion, explainability, dashboard.

We picked three parallel, structurally different detectors instead of one larger model because each catches a different attack family, and a failure in one doesn't take the others down with it. The outputs of all three — rule flags, tabular anomaly score, sequence anomaly score — are combined with the 27 engineered features (Section 1.4) and fed into a single XGBoost classifier (max depth 5, 300 estimators, 8-class multi:softprob objective covering normal plus the seven attack types). That fusion model produces the final risk score, calculated as 100 × (1 − P(normal)), and the attack-type label.

We chose a GRU over an LSTM-autoencoder for two reasons: it trains faster on CPU, and “the actual next action wasn't in the model's top 5 predictions” is a much easier thing to explain to an analyst than a reconstruction-error threshold.

1.4 Feature Engineering

27 features are computed per event and fed to the tabular ensemble and the fusion classifier. The raw command sequence is not one of these — it's fed separately, as a token sequence, into the GRU.

Temporal (7)

Feature	Definition
hour_sin	sin (2π × hour / 24)
hour_cos	cos (2π × hour / 24)
day_of_week	0–6
is_weekend	binary
is_work_hours	relative to the entity's own profile
time_since_last_access	seconds since the entity's previous event
session_frequency_1h	rolling count of sessions in the last hour

Behavioral (10)

Feature	Definition
auth_failure_rate_10min	failed / total attempts in the last 10 minutes

Feature	Definition
resource_diversity_24h	unique resources touched in the last 24 hours
resource_novelty	count of resources never accessed before by this entity
session_duration_zscore	(x − entity mean) / entity std. dev.
bytes_transferred_zscore	(x − entity mean) / entity std. dev.
auth_method_mismatch	binary — new auth method for this entity?
command_entropy	Shannon entropy of the command distribution
unique_entities_per_source_ip_1h	credential-stuffing signal
is_off_hours_access	binary
privilege_expansion_rate	new resources / total resources ever accessed

Geo-Spatial (5)

Feature	Definition
geo_velocity_kmh	haversine distance / time difference between consecutive logins
impossible_travel_flag	binary — velocity exceeds 900 km/h
geo_distance_from_primary	haversine distance to the entity's home location
is_new_geo	binary — location never seen before for this entity
geo_anomaly_score	weighted combination of the above

Device (3)

Feature	Definition
fingerprint_mismatch	known device_id, but OS/MAC differs from history
new_device_flag	binary — fingerprint never seen before
protocol_mismatch	unusual protocol for this entity_type

Cold-Start Meta (2)

Feature	Definition
entity_maturity_weight	min (1.0, entity_event_count / 20)
is_cold_start	binary — maturity weight below 0.5

1.5 Cold-Start Handling

New entities have no history to compare against, so scoring them the same way as an established account either means flagging everything they do (alert fatigue) or trusting them completely (a blind spot). We handle this with cohort blending: a new entity's early events are scored primarily against the baseline of its peer group — same entity_type (user, device, or service account) — rather than its own empirical history, which doesn't exist yet.

The blend is continuous rather than a tiered if/else: weight = min (1.0, entity_event_count / 20). At zero events, weight is 0 and the entity is scored purely against its cohort; by 20 events, weight reaches 1.0 and the entity is scored purely against its own accumulated profile. Entities with a maturity weight below 0.5 are visibly flagged on the dashboard as low confidence / new entity, so an analyst knows to apply more scrutiny rather than trusting the score at face value.

1.6 Concept Drift Handling

Legitimate behavior changes over time — a new project, a role change, a new device — and a static baseline would keep flagging that as anomalous indefinitely. Each entity's profile is updated with an exponentially weighted moving average, decay rate 0.05, so the baseline itself shifts to follow real, sustained behavior change within roughly 20 events.

On top of that, we run River's ADWIN (Adaptive Windowing, delta = 0.002) on each cohort's rolling anomaly rate. When ADWIN detects a statistically significant shift in that rate, it temporarily widens the cohort's acceptance window and logs a drift event to the dashboard, rather than silently suppressing or silently accepting the new pattern.

1.7 Label Handling

Ground-truth labels are generated and stored in a separate file (hidden_labels.csv), joined back to the event stream only by event_id, and only for evaluation. They never enter the feature pipeline or reach any detector — nothing in the detection path can see the answer while it's scoring an event.

1.8 Serving Architecture

The dashboard was originally planned as a Streamlit app, since it's fast to build. Partway through the build we moved off it: Streamlit blocks its main thread during model inference, which would mean the dashboard freezing during scoring and made concurrent access by multiple analysts impractical. We rebuilt the serving layer as a FastAPI (ASGI) backend that handles the XGBoost inference and data routing asynchronously, with a lightweight vanilla JavaScript frontend consuming it. That's a more realistic base for how this would actually need to run — able to serve several analysts at once without one person's request blocking another's.

2. Metrics

2.1 Dataset Composition

Property	Value
Total events	208,734
Total entities	600 (400 users, 150 edge devices, 50 service accounts)
Date range	Jan 1, 2025 – Mar 2, 2025 (61 days)
Attack rate	1.47% of events (≈ 3,068 events)
Normal rate	98.53% of events

2.2 Evaluation Methodology

We used a strict temporal split rather than a random one, specifically so the model can't train on anything from the future. The cutoff is anchored to a fixed 30-day window from the first timestamp in the dataset, not a fixed ratio of the data: everything from Jan 1–30, 2025 is the training set (139,454 events), and everything from Jan 31 onward — the remaining 31 days, through Mar 2 — is the test set (69,280 events). The test window ends up slightly longer than the training window because the cutoff is a fixed date, not a fixed split ratio; we kept it that way rather than trimming the test set, since the goal was validating on every event the model hadn't seen, not matching a round split ratio.

All figures below come from that held-out 69,280-event test set.

2.3 Headline Results

The evaluation criteria call out precision at a realistic analyst alert budget as the metric that matters most: sort alerts by risk score and check how many of the top 1% are real. That's the number we lead with.

Metric	Value	Note
Precision @ Top 1%	99.71%	Of the highest-risk-scored 1% of test events, 99.71% were genuine anomalies.
PR-AUC	93.91%	The right summary metric given the class imbalance here; ROC-AUC alone would look inflated by the size of the normal class.
Macro F1	91.81%	Averaged across all 8 classes equally, so rare classes count as much as common ones.
Macro Precision	91.26%	
Macro Recall	92.57%	
Overall Accuracy	99.59%	Inflated by the size of the normal class; included for completeness, not as a headline number.
ROC-AUC	99.12%	Derived from the true/false positive rate curve.
False Positive Rate (normal class)	0.24%	165 of 68,265 normal events misclassified as some anomaly type. See Section 2.5.
Inference time	≈ 14.2 ms / event	FastAPI async inference.
Training time	≈ 47 seconds	Full pipeline: tabular ensemble + GRU + XGBoost fusion, trained in parallel.

2.4 Per-Class Performance

Class	Precision	Recall	F1	Support
normal	99.85%	99.76%	99.80%	68,265
brute_force	100.00%	100.00%	100.00%	151
impossible_travel	97.14%	100.00%	98.55%	136
credential_stuffing	100.00%	100.00%	100.00%	143
lateral_movement	96.91%	93.45%	95.15%	168
device_spoofing	100.00%	100.00%	100.00%	68
low_and_slow	100.00%	100.00%	100.00%	144
insider_drift	36.19%	47.32%	41.01%	205

The six “loud” attack types — brute_force, impossible_travel, credential_stuffing, lateral_movement, device_spoofing, low_and_slow — all score 95% F1 or higher, most at a clean 100%. insider_drift is the clear outlier and is addressed directly in Section 3.1.

2.5 Confusion Matrix

Rows are actual class, columns are predicted class. N = normal, BF = brute_force, IT = impossible_travel, CS = credential_stuffing, LM = lateral_movement, DS = device_spoofing, LS = low_and_slow, ID = insider_drift.

Actual \ Pred.	N	BF	IT	CS	LM	DS	LS	ID
N	68,100	0	4	0	1	0	0	160
BF	0	151	0	0	0	0	0	0
IT	0	0	136	0	0	0	0	0
CS	0	0	0	143	0	0	0	0
LM	11	0	0	0	157	0	0	0

Actual \ Pred.	N	BF	IT	CS	LM	DS	LS	ID
DS	0	0	0	0	0	68	0	0
LS	0	0	0	0	0	0	144	0
ID	104	0	0	0	4	0	0	97

Nearly all of the system's confusion lives in one place: 160 of the 165 total false positives on the normal class are normal events mis-tagged as insider_drift, and 104 of the 205 true insider_drift events in the test set were missed entirely (scored as normal). Every other class is confused with at most one or two others, and only in single digits.

3. Known Limitations

3.1 Insider Drift Detection

insider_drift is the weakest-performing class in the system, and it's worth being direct about why. Precision is 36.19% and recall is 47.32% (F1 0.41) — the model misses more than half of true insider-drift events, and when it does raise this specific label, it's wrong about two-thirds of the time. Almost all that error is a two-way confusion with the normal class: 104 of 205 true insider-drift events score as normal (missed), and 160 of 68,265 normal events score as insider-drift (false alarm).

This is partly by design and partly a genuine limitation. insider_drift is defined as a slow, 14–30 day shift in working hours or resource footprint — the same shape as a lot of completely legitimate change, like a promotion or a new project. We didn't tune the detection threshold aggressively on this class, because doing so would pull more of that legitimate drift into the alert queue, and alert fatigue is a real cost for a SOC analyst. What's left is a class the system genuinely struggles to separate cleanly from normal, not a case where we hit a target number and stopped. Closing this gap further would likely need either more training examples of this specific pattern, or a different detection approach altogether — comparing an entity's trajectory over weeks rather than scoring individual events in isolation.

3.2 Concept-Drift Sensitivity for Sparse Entities

ADWIN needs a minimum run of events before it can reliably establish a rolling baseline and detect a shift in it. Entities that check in rarely — some of the edge-device population in particular — accumulate that history more slowly, so drift correction for them lags drift correction for an actively-used account. In practice, a rarely active device that changes behavior will stay on a stale baseline longer than a busy one before the system adjusts.

3.3 Heuristic Cold-Start Weighting

The cohort-blending weight (min (1.0, n/20)) is a fixed formula we chose for interpretability and because it was fast to implement correctly under time pressure — not because 20 events is a proven optimal ramp-up point. It's likely too fast for some entity types and too slow for others; a more rigorous version would learn the ramp rate per entity_type or per cohort, rather than applying the same curve to a user, a service account, and an edge device alike.

3.4 Synthetic-Data-Only Validation

Every number in this report comes from evaluating against our own synthetic dataset, where we control the ground truth. That's necessary — real intrusion datasets with reliable labels are scarce and mostly unavailable for a hackathon — but it also means these numbers describe how well the system separates the specific patterns we designed, not how it would perform against a real enterprise's traffic. A production deployment would need validation against real logs, which will have more entity_type variety, messier resource naming, and legitimate anomalies (system migrations, outages, mass password resets) that this generator doesn't model.

3.5 Production Gaps / Future Scope

A few things describe the intended production path but weren't built for this submission:

- Streaming ingestion — the system currently scores a batch CSV replay through the FastAPI service, not a live Kafka or WebSocket stream. In production, ingestion would run through Kafka with per-entity profiles cached in Redis, so a scoring request doesn't reconstruct an entity's history from scratch every time.
- Alert persistence — alerts currently live in memory / the active session; a production version would persist them to a store like ClickHouse for historical query and audit.
- Lateral-movement modeling — the GRU next-action model works at the level of individual action sequences. A graph-based approach, mapping entity-to-resource relationships and using something like GraphSAGE, would represent the actual network topology more directly and should generalize better to attack paths the sequence model hasn't seen the shape of before.